SG

SiteGit

FIELD VERSION CONTROL

TEAM DISCUSSION DRAFT · v0.1

다중 건설장비 작업을 위한 Branch·Commit·Merge 운영 구조

여러 굴착기가 동일 공정을 나누어 수행하고, 장비별 작업 Commit을 센서 Evidence로 검증한 뒤 하나의 작업 Branch로 통합하여 공식 현장 상태인 Main에 반영하는 구조에 대한 팀 논의안

구현 확정안 아님 · 팀 합의 후 계약과 데이터 모델 확정

문서 목적	대상	작성일
구조 합의 및 구현 범위 결정	Backend · Frontend · Isaac Adapter	2026.08.18

1. 논의 배경과 공통 전제

01 · CONTEXT

SiteGit의 핵심은 작업 결과를 바로 공식 현장 상태로 덮어쓰지 않고, 검증 가능한 Commit으로 남긴 뒤 관리자가 승인한 결과만 Main에 Merge하는 것이다.

데이터 전제

현재 화면에 표시된 사상구 정비사업 목표량·잔여량·진척률은 UI 확인을 위한 임시값이다. 실제 데이터 계약과 산정 방법은 추후 교체하며, 이번 논의에서는 작업 Version 구조와 승인 흐름에 집중한다.

CURRENT FOUNDATION

현재 확보된 기반

- 웹 자연어 Work Package 생성
- Backend 작업지시와 Isaac Adapter 실행
- 카메라·LiDAR·IMU 작업 전후 Evidence
- Execution Commit 및 관리자 승인·반려
- 승인된 Commit만 공식 웹 맵에 반영

CURRENT LIMIT

현재 구조의 한계

- 장비 한 대의 Commit이 곧바로 Main 승인 대상
- 동일 공정을 여러 장비가 나누는 개념이 없음
- 장비별 부분 작업량과 공간 중첩 검증 부족
- 작업 단계별 Branch와 최종 통합 Merge가 없음
- Commit 기준 현장 타임라인 재생 기능이 없음

현재

Work Instruction → 장비 실행 → Commit → 관리자 승인 → Main

단일 장비 데모에는 적합하지만 다중 장비 협업과 공정 단위 인수에는 부족하다.

→

제안

Task Branch → 장비별 Commit → 통합 검증 → Branch 승인 → Main

장비별 책임과 Evidence를 보존하면서 공정 전체를 하나의 결과로 인수한다.

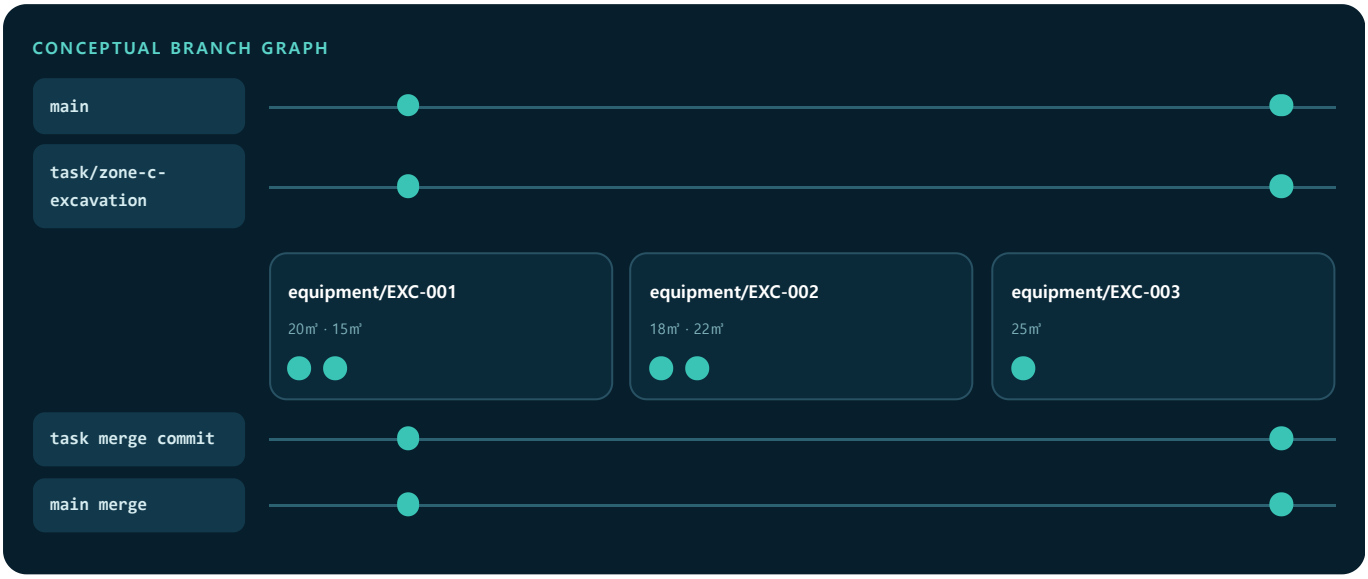
이번 구조가 해결해야 하는 질문

1. 같은 구역을 여러 굴착기가 작업할 때 각 결과를 어떻게 구분하고 합칠 것인가?
2. 각 장비 Commit은 언제 Clean, Warning, Conflict인가?
3. 공정 전체가 어느 상태일 때 Main에 Merge할 수 있는가?
4. 굴착·관로 설치·되메우기처럼 순서가 있는 공정을 어떻게 Version으로 연결할 것인가?

2. 제안하는 Branch 계층

02 · BRANCH

Main은 공식 승인 현장 상태이며, 공정 단위 Task Branch 아래에서 장비별 실행 흐름이 독립적인 Commit을 생성한다.



MAIN

공식 현장 상태

관리자가 최종 인수한 작업만 존재한다. 웹의 기본 3D 맵과 공식 타임라인은 Main을 기준으로 렌더링한다.

TASK BRANCH

공정 단위 변경 집합

예: Zone C 토사 굴착. Work Package 목표, 공간 경계, 허용오차, 총작업량과 완료 조건을 소유한다.

EQUIPMENT LANE

장비별 실행 흐름

각 굴착기가 생성한 Commit을 장비 ID별로 묶어 보여주는 실행 단위다. 실제 Git 구현에서는 별도 Branch ref 또는 Commit metadata로 표현할 수 있다.

TASK MERGE COMMIT

통합 작업 결과

모든 장비 Commit을 합성하고 충돌을 해소한 공정 결과다. 관리자는 이 통합 Diff와 Evidence를 보고 Main Merge를 결정한다.

용어 주의

"Branch 안의 Branch"는 사용자 화면의 계층 표현이다. 저장 계층에서는 모든 Commit을 하나의 DAG로 관리하고 `task_branch_id`, `equipment_id`, `parent_commit_ids`로 관계를 표현할 수 있다.

제안 기본 원칙

- 장비 Commit은 Main을 직접 변경하지 않는다.
- 장비 Commit 검토와 Task Branch 최종 Merge 검토는 서로 다른 단계다.
- 반려된 Commit은 삭제하지 않고 해당 장비 흐름의 Conflict로 보존한다.
- Task Branch가 완료되기 전에도 통합 예상 Diff는 미리 볼 수 있어야 한다.

3. 다중 장비 작업 페루프

03 · WORKFLOW

작업 발행부터 Main Merge까지 모든 단계는 같은 Work Package·Branch·Commit 식별자로 추적한다.

1. **공정 Work Package 생성** — DRAFT
   관리자가 Zone, 작업 종류, 목표 형상·물량, 완료 조건을 입력
2. **Task Branch 생성** — PLANNED
   Main의 특정 Commit을 기준점으로 `task/zone-c-excavation` 생성
3. **장비·공간·물량 배분** — ASSIGNED
   EXC-001/002/003에 서로 다른 작업 Polygon과 목표량 할당
4. **Isaac Sim 실행** — EXECUTING
   각 장비가 할당 구역에서 작업하고 Stage 결과를 실제로 읽음
5. **장비별 Execution Commit** — COMMITTED
   카메라·LiDAR·IMU, 수행시간, 작업량, Height Map·Mesh Diff 저장
6. **Commit 검증·Branch 통합** — INTEGRATING
   공간 중첩, 물량 중복, 경계 초과, Evidence 누락을 판정
7. **Task Merge Request** — REVIEW
   전체 장비 결과를 합친 통합 지형 Diff와 공정 완료 조건 생성
8. **관리자 Main Merge** — MERGED
   승인 시 공식 맵·Version Timeline 갱신, 반려 시 Conflict 유지

Commit 하나에 포함할 최소 정보

구분	필수 데이터	저장 목적
식별	Task Branch, Work Package, 장비, Parent Commit	책임과 계보 추적
작업	작업 종류, 구역, 시간, 수행량, 장비·Tool Pose	실행 내용 검증
지형	작업 전후 Height Map, Point Cloud, Mesh/voxel delta	절토·성토와 형상 Diff
센서	카메라, LiDAR, IMU, 타임스탬프, 품질 지표	관리자 Evidence
판정	Clean/Warning/Conflict, 규칙 결과, 검토자 의견	Merge Gate와 감사 이력

현재 데모 범위

Isaac Sim 재시작 시 Stage가 원본 USD로 돌아가는 것은 현재 단계에서 허용한다. Version Timeline과 공식 맵은 저장된 Commit 자료로 구성하며, 승인 상태를 Isaac Stage에 재적용하는 기능은 실제 상태 저장 단계에서 별도로 논의한다.

4. Merge 및 Conflict 판정 제안

04 · MERGE

개별 Commit이 정상이어도 여러 장비 결과를 합쳤을 때 충돌할 수 있으므로, 장비 Commit Gate와 Task Branch Gate를 분리한다.

판정	예시 조건	시스템 동작
Clean	할당 Polygon 내부, 다른 장비 결과와 비중첩, 필수 Evidence 완비, 목표 오차 이내	Task Branch에 자동 통합 가능
Warning	경계 인접, 일부 센서 품질 저하, 작업량 편차, 연결부 Height 차이	통합 후보로 유지하고 관리자 확인 요청
Conflict	동일 토사 중복 계산, 공간 중첩, 계획 경계 초과, 공정 순서 위반, Evidence 필수값 누락	Branch Merge 차단, 재작업 Commit 요구

필수 Merge 검사 항목

① 공간 충돌

장비별 작업 Polygon, 안전반경, 작업 후 지형 영역이 겹치는지 검사한다.

② 물량 중복

같은 voxel/height cell의 절토량이 장비별 Commit에서 중복 집계되는지 확인한다.

③ 형상 연속성

장비 구역 경계에서 높이·경사·관로 연결부가 허용오차 내인지 확인한다.

④ 공정 의존성

굴착 완료 전 관로 설치, 관로 승인 전 되메우기 같은 순서 위반을 차단한다.

⑤ Evidence 품질

카메라 전후, LiDAR Point Cloud, IMU, 시간 순서와 좌표계 일치 여부를 검사한다.

⑥ 목표 충족

Task Branch 전체 작업량과 최종 형상이 Work Package의 완료 조건을 만족하는지 검사한다.

권장 2단계 승인

단계	검토 대상	결과
장비 Commit 검토	해당 장비의 작업 전후 Evidence와 부분 Diff	Task Branch 통합 / 재작업 / Conflict
Task Branch 검토	모든 장비를 합친 최종 지형, 총작업량, 통합 Conflict	Main Merge / Branch 반려

구현 원칙

판정 규칙은 코드에 고정하지 않고 프로젝트별 정책으로 분리한다. 같은 공간 중첩도 측량 오차인지 실제 충돌인지에 따라 기준이 달라질 수 있으므로 허용오차와 필수 센서 목록은 Work Package 또는 Merge Rule Version에 귀속한다.

5. 공식 맵 Timeline과 Diff UX

05 · TIMELINE

Main에 Merge된 공정 Branch를 큰 단계로 보여주고, 각 단계 내부에서 장비별 Commit을 세부 이력으로 탐색한다.



CURRENT MODE

현재 맵

Main의 최신 승인 상태를 표시한다. 운영자가 기본으로 보는 화면이다.

TIMELINE MODE

변화 재생

슬라이더를 움직이면 Baseline부터 선택 Commit까지의 변경을 재구성한다.

DIFF MODE

두 Version 비교

기준 Commit과 비교 Commit 사이의 절토·성토·객체 변경만 강조한다.

BRANCH MODE

승인 전 미리보기

Main과 Task Branch의 예상 결과를 비교하지만 공식 맵에는 적용하지 않는다.

성능을 위한 저장 전략

- **Baseline + Delta:** 각 Commit에는 전체 맵 대신 추가·삭제·이동 Prim과 지형 변화량을 저장한다.
- **주기적 Snapshot:** 일정 Commit 또는 공정 Merge마다 전체 Height Map/Mesh Snapshot을 생성한다.
- **근접 Snapshot 재생:** 슬라이더 이동 시 가장 가까운 Snapshot부터 필요한 Delta만 적용한다.
- **대용량 Evidence 분리:** Point Cloud·이미지·Mesh 파일은 SSD에 저장하고 DB에는 storage key와 metadata만 둔다.

화면에서 Commit 선택 시 보여줄 정보

요약	지형·객체	작업 Evidence	판정
Branch, Commit, 장비, 작업자, 시간	절토·성토량, 변경 영역, 관로·토사 객체	카메라 전후, LiDAR, IMU, 수행시간	Clean/Warning/Conflict와 관리자 의견

공식 Timeline 규칙

승인 전 장비 Commit은 Main Timeline에 포함하지 않는다. Task Branch가 최종 승인된 시점에 하나의 Main Merge Commit으로 들어가며, 세부 화면에서 포함된 장비 Commit을 펼쳐본다.

6. 팀에서 결정해야 할 항목

06 · DECISIONS

아래 항목을 먼저 합의한 뒤 API 계약과 DB 모델을 확정해야 Frontend, Backend, Isaac Adapter를 병렬 개발할 수 있다.

1	**Task Branch의 생성 단위** Zone 전체, 공정 단계, 또는 Work Package 하나 중 무엇을 Branch로 볼 것인가?	권장: Zone + 공정 단계
2	**장비 Commit 생성 주기** 버킷 1회, 일정 물량, 일정 시간, 작업 세션 종료 중 어느 단위를 사용할 것인가?	권장: 의미 있는 작업 Chunk 또는 세션 종료
3	**장비 흐름의 저장 방식** 실제 하위 Branch ref로 만들지, Task Branch 내부 equipment lane metadata로 표현할지 결정	권장: 초기에는 lane metadata
4	**2단계 승인 권한** 장비 Commit 통합 승인자와 최종 Main Merge 승인자를 같은 역할로 둘 것인가?	권장: 데모는 동일 관리자, 이후 역할 분리
5	**지형 Version 표현** Height Map, voxel, full mesh, vertex delta 중 공식 Diff와 재생의 기준을 무엇으로 할 것인가?	권장: Height Map + Mesh artifact 병행
6	**Conflict 허용오차** 공간 중첩, 높이 차, 작업량 편차, 센서 누락 기준을 프로젝트별로 어떻게 Version화할 것인가?	권장: Merge Rule YAML/JSON
7	**Main Timeline 표시 단위** 모든 장비 Commit을 노출할지, Task Merge만 기본 표시하고 세부 펼침을 제공할지 결정	권장: Task Merge 기본 + 장비 Commit 펼침
8	**Isaac 재시작 상태** 현 단계처럼 원본 USD로 시작할지, 마지막 Main 상태를 Stage에 복원할지 결정	권장: 현재는 초기화, 운영 단계에서 복원

논의 시 확인할 예시 시나리오

Zone C 토사 굴착 — 굴착기 3대

EXC-001, 002, 003이 서로 다른 작업 영역과 물량을 배정받는다. 각 장비가 1~2개의 Commit을 제출하고, EXC-002의 한 Commit은 EXC-001 영역과 중첩되어 Conflict가 된다. 재작업 Commit으로 충돌을 해소한 뒤 모든 장비 결과를 Task Branch에서 통합하고, 관리자가 통합 Height Map·센서 Evidence를 확인하여 Main에 Merge한다.

7. 단계별 구현 제안

07 · ROADMAP

팀 합의 전에 전체 구조를 한 번에 구현하지 않고, 계약 → 다중 장비 Commit → Merge Engine → Timeline 순서로 확장한다.

PHASE 1

계약·모델

- Branch/Commit/Merge Request 스키마
- 상태 전이
- API 계약 테스트

PHASE 2

다중 장비

- 장비별 할당 영역
- 개별 Evidence Commit
- Task Branch UI

PHASE 3

Merge Engine

- 공간·물량 충돌
- 2단계 승인
- 통합 Diff

PHASE 4

Timeline

- Snapshot + Delta
- Commit Slider
- Branch Diff

현재 코드에서 재사용할 수 있는 요소

현재 요소	확장 방향
`WorkInstruction`	Task Branch와 장비 할당 정보를 참조하도록 확장
`ExecutionCommit`	Parent Commit, equipment_id, terrain delta, branch_id 추가
카메라·LiDAR·IMU Evidence	장비 Commit의 필수 Artifact와 품질 Gate로 재사용
Trust Gate 승인·반려	장비 Commit Gate와 Task Merge Gate로 분리
`GET /isaac/map-objects`	Main의 선택 Version 또는 Timeline Commit 기준 조회로 확장
3D Web Map	Current/Timeline/Diff/Branch 모드 추가

팀 합의 후 가장 먼저 만들 산출물

1. **상태 전이 문서:** Task Branch, Equipment Commit, Merge Request 상태와 허용 전이
2. **JSON Schema:** Branch, Commit, Terrain Delta, Merge Evaluation 계약
3. **단일 통합 테스트:** 굴착기 2대가 Commit → 한 건 Conflict → 재작업 → Task Merge → Main 반영
4. **UI Wireframe:** Branch 목록, 장비 Commit lane, 통합 Diff, Timeline Slider

제안 결론

SiteGit은 장비 한 대의 작업 기록 시스템이 아니라, 여러 장비가 만든 물리적 변경을 Version으로 통합하고 사람이 Evidence를 근거로 공식 현장 상태에 Merge하는 시스템으로 정의한다. 구현의 핵심은 "장비 Commit은 Main을 직접 바꾸지 않는다"는 불변식을 유지하는 것이다.